

Multi-Camera Video Processing & V2X Pipeline

1 Pipeline Architecture

This document details the architecture and mathematical operations of the `process_video.py` pipeline. The system takes multi-camera video feeds, performs YOLO-based object detection, projects 2D pixels into 3D world coordinates, converts these local coordinates into global GPS locations, deduplicates objects across overlapping camera views, and uploads the data to a V2X (Vehicle-to-Everything) API.

1.1 VideoObjectDetector

Responsible for processing individual camera streams.

- **Detection:** Runs YOLOv8 to identify objects (e.g., persons, vehicles) and extracts their 2D bounding boxes and bottom-center pixel coordinates (u, v) .
- **Spatial Mapping:** Uses the camera intrinsic matrix (K) , distortion coefficients, height, pitch, and yaw to cast a ray from the pixel to the ground plane, yielding local 3D coordinates (X, Z) .
- **Global Mapping:** Rotates the (X, Z) coordinates by the camera's true geographical heading and translates them to GPS Latitude and Longitude.

1.2 MultiCameraPipeline

Responsible for aggregating data from multiple `VideoObjectDetector` instances.

- **Synchronization:** Reads frames from multiple video streams concurrently.
- **Spatial Deduplication:** Merges detections of the same object type from different cameras if their geographic distance (calculated via Haversine formula) is below a specific threshold (e.g., 8 meters).
- **Temporal Tracking:** Maintains a global track history to assign consistent IDs across frames.

2 Mathematical Models and Formulas

2.1 2D Pixel to Local 3D Coordinates

The script first undistorts the raw pixel (u, v) using the distortion coefficients. It then projects it into a normalized camera ray:

$$\vec{r}_{cam} = \begin{bmatrix} \frac{u_{undist} - c_x}{f_x} \\ \frac{v_{undist} - c_y}{f_y} \\ 1 \end{bmatrix}$$

This ray is rotated using the Pitch (θ_x) and Yaw (θ_y) matrices to align with the world:

$$\vec{r}_{world} = R_y R_x \cdot \vec{r}_{cam} = \begin{bmatrix} d_x \\ d_y \\ d_z \end{bmatrix}$$

To find the intersection with the ground, the script calculates a scaling factor t based on the camera height H . In OpenCV coordinates, Y points down, so the ground is at $Y = H$.

$$t = \frac{H}{d_y}$$

$$X = t \cdot d_x$$

$$Z = t \cdot d_z$$

Note: If $d_y \leq 0$, the ray is pointing at or above the horizon, and no ground intersection is computed.

2.2 Local 3D to Global GPS Coordinates

Once the local (X, Z) distances (in meters) are found, they must be converted to global GPS coordinates relative to the camera pole's origin $(Lat_{origin}, Lon_{origin})$ and the camera's Heading (ψ).

First, the local coordinates are rotated by the heading angle to align with True North (Northing) and East (Easting):

$$\text{Easting} = Z \cdot \sin(\psi) + X \cdot \cos(\psi)$$

$$\text{Northing} = Z \cdot \cos(\psi) - X \cdot \sin(\psi)$$

Next, the flat-earth approximation is used to convert meter offsets into degrees of latitude and longitude. The standard conversion is 111,320 meters per degree of latitude.

$$\text{MetersPerDeg}_{lat} = 111320.0$$

$$\text{MetersPerDeg}_{lon} = 111320.0 \cdot \cos(Lat_{origin})$$

$$Latitude = Lat_{origin} + \frac{\text{Northing}}{\text{MetersPerDeg}_{lat}}$$

$$Longitude = Lon_{origin} + \frac{\text{Easting}}{\text{MetersPerDeg}_{lon}}$$

2.3 Object Deduplication (Haversine Distance)

To prevent counting the same object twice when it appears in overlapping camera views, the pipeline calculates the great-circle distance between two GPS points (Lat_1, Lon_1) and (Lat_2, Lon_2) using the Haversine formula:

$$dLat = Lat_2 - Lat_1$$

$$dLon = Lon_2 - Lon_1$$

$$a = \sin^2\left(\frac{dLat}{2}\right) + \cos(Lat_1) \cdot \cos(Lat_2) \cdot \sin^2\left(\frac{dLon}{2}\right)$$

$$c = 2 \cdot \arcsin(\sqrt{a})$$

$$\text{Distance} = R_{earth} \cdot c$$

Where $R_{earth} \approx 6,371,000$ meters. If the distance is less than the merge radius (e.g., 8 meters), the detections are merged, keeping the one with the higher confidence score.